Assignment 1: Critique

Muskaan Kumar (mk10608)

Collaboration and AI use statement

I used Claude (Anthropic) as an AI assistant to help me with some portions of the assignment. I used it to help me understand the codebase structure when I could not follow through some parts, verify that my trace through the hot path execution was correct, identify and confirm design decisions, and draft the modified code files for my proposed fixes. I wrote the critique document (passed it through AI for final grammatical check) and used Carl Cook's CppCon 2017 talk as a reference to help make design decisions. All final writing and understanding is my own. I did not collaborate with other students. I have attached a link to a doc I maintained throughout the assignment where I jotted down my experiments and understanding.

1. Describe one possible execution of the hotpath.

A market data message arrives and the capture tap records a wall-clock entry time. `OnMarketData` decodes the header via `memcpy` into a packed struct, checks the sequence number for gaps, and applies the update to the order book. The book maintains a cached "top of book" (best bid/ask price and size), so reading it is a direct field access. `Requote` then checks for a halt (sequence gap or loss limit) and book validity (both sides present, not crossed). If both pass, it computes the microprice (a size-weighted average of the best bid and ask), calculates the credit (a spread that widens with volatility), and determines a quote price on each side. For each side, it checks the position limit and whether the resting quote already matches. If a new quote is needed, `SendQuote` logs the decision, appends to a history vector, and calls `Send`. Finally, the handler records its completion time in a preallocated latency log. The entire path runs single-threaded with no system calls or heap allocation in steady state.

2. Describe a flaw in the evaluation and a potential fix.

What is the evaluation doing?

The evaluation computes only the arithmetic mean of all callback latencies. It sums every end-to-end latency and divides by the count.

Why is that a problem?

A single outlier can dominate the mean. In `basic.scn` the first callback takes ~3,800 ns because of cold instruction caches, cold data caches, and an untrained branch predictor. Steady-state callbacks sit near the median of 2,458 ns. With only 54 messages, that cold-start spike inflates the average substantially. Worse, the mean cannot distinguish "consistently slow" from "fast with rare spikes"—these require completely different fixes, but the mean makes them look the same.

What is the fix?

Report distributional statistics: median, p85, p90, p99, min, and max. The modified `analysis.h` collects individual latencies into a sorted vector and computes percentiles:

```
// analysis.h -- CHANGED: collect + sort + report percentiles
std::vector<double> latencies;
latencies.reserve(capture.deliveries.size());
// ... collect individual latencies ...
std::sort(latencies.begin(), latencies.end());

auto percentile = [&](double p) -> double {
    auto idx = static_cast<std::size_t>(
        std::ceil(p * static_cast<double>(count))) - 1;
    if (idx >= count) idx = count - 1;
    return latencies[idx];
};

out << "average latency: " << mean << " ns\n";
out << "median  latency: " << percentile(0.50) << " ns\n";
out << "p85     latency: " << percentile(0.85) << " ns\n";
out << "p90     latency: " << percentile(0.90) << " ns\n";
out << "p99     latency: " << percentile(0.99) << " ns\n";
```

Why does this matter?

With percentile reporting on the modified code, `basic.scn` shows median 92 ns versus mean 490 ns—the mean is 5× the median, entirely due to cold-start callbacks. The p85 sits around the steady-state cluster, confirming that 85% of callbacks are fast and the mean was lying. Without percentiles, a developer would chase a "3,174 ns average" that does not represent any real callback.

How does this relate to Carl Cook's CppCon 2017 critique?

Cook says microbenchmarks are the wrong way to *measure*—they isolate a function in a tight loop with hot caches, hiding the real cost. The evaluation already avoids that mistake: it measures end-to-end inside a full simulation. The flaw is in how those correct measurements are *reported*: mean-only reporting hides the distribution. Percentiles and Cook's critique are complementary—one fixes the measurement, the other fixes the summary.

Comparison: unmodified code, then modified code

Scenario	Msgs	Median (ns)	p90	p99	Max	CV%
basic	54	2,458	3,275	3,717	3,845	16.8

Table 1. Unmodified code with percentile reporting. The median is below the mean because outliers inflate the average. CV% (coefficient of variation) shows how widely the distribution spreads.

Scenario	Msgs	Mean (ns)	Median (ns)	Min	Max
basic	54	490	92	54	10,499

Table 2. Modified code (all three hot-path fixes applied). The median of 92 ns reveals the true steady-state cost. The max (10,499 ns) is a single cold-start callback, and it drags the mean (490 ns) up.

3. Describe a good design decision in the market maker.

What does the code do?

The order book in `book.h` stores orders in a flat array indexed by the exchange-assigned reference number. Since refs are assigned sequentially starting from 1, the array is dense. Looking up order ref 42 means accessing `orders_[42]`—a single indexed memory load.

Why is this a good decision?

The alternative would be a hash map like `std::unordered_map`. A hash map lookup requires: computing a hash of the key, indexing into the bucket array, following a pointer to a heap-allocated node (almost always a cache miss because nodes are scattered in memory), and comparing the key. This costs roughly 20–25 ns per lookup. (modified code to include unordered map implemetation to test these numbers)

The flat array costs ∼3–5 ns: one indexed load, likely a cache hit because nearby refs are accessed together as messages arrive in order. The book looks up an order on nearly every incoming market data message—adds, executions, cancels, deletes, replaces all identify orders by ref. A 5× speedup on the most frequent hot-path operation is significant. The tradeoff is memory: the array is sized for up to a million refs (16 MB), allocated once at startup. In a latency-sensitive system, 16 MB of memory is a trivial cost for deterministic, cache-friendly access.

4. Describe a flaw in the market maker and a potential fix.

What is happening?

The risk checks use C++ exceptions as control flow. When the position limit is reached, `CheckPosition` in `risk.h` throws a `RiskViolation`. The caller in `market_maker.cpp` catches it and silently returns.

```
// risk.h -- ORIGINAL: throws on position breach
static void CheckPosition(sim::Side side, std::int64_t position) {
    const bool breach = side == sim::Side::Buy
        ? position + kQuoteShares > kPositionLimit
        : position - kQuoteShares < -kPositionLimit;
    if (breach) {
        throw RiskViolation(RiskError::PositionLimit, "position limit");
    }
}
```

```
// market_maker.cpp -- ORIGINAL: catch and return
try {
    RiskGate::CheckPosition(side, orders_.position());
} catch (const RiskViolation&) {
    return;
}
```

Why is this a problem?

The design document argues exceptions are "zero-cost when not thrown," but the position limit is hit routinely during normal trading as inventory accumulates. Each throw costs ~1,500 ns: the runtime allocates the exception object (including a string), walks the stack to find the catch handler, runs destructors, and transfers control. The exception is doing the work of a simple boolean check.

What is the fix?

Add a non-throwing boolean method using the same comparison, and replace try-catch with if-return:

```
// risk.h -- ADDED: boolean alternative
static bool PositionBlocked(sim::Side side, std::int64_t position) {
    return side == sim::Side::Buy
        ? position + kQuoteShares > kPositionLimit
        : position - kQuoteShares < -kPositionLimit;
}
```

```
// market_maker.cpp -- CHANGED: if-return instead of try-catch
if (RiskGate::PositionBlocked(side, orders_.position())) {
    return;
}
```

One branch (~3 ns) replaces one throw (~1,500 ns). The boolean comparison is identical to the one inside `CheckPosition`—same logic, same outcome. The throwing version is kept for code outside the hot path. `make verify` passes: the strategy produces identical orders. This design choice was inspired by Carl Cook's talk.

5. Describe a second flaw in the market maker and a potential fix.

What is happening?

The decision log calls `fprintf` to format every order *before* transmitting it—seven format conversions including two 64-bit integer divisions, directly on the hot path between decision and send.

```
// decision_log.h -- ORIGINAL: formats on every call
void Record(sim::Timestamp now, const char* action,
            sim::Side side, sim::Price price,
            uint32_t shares, sim::Token token) {
    std::fprintf(file_,
        "%llu %s %c %lld.%04lld x %u token=%llu\n",
        (unsigned long long)now, action, (char)side,
        (long long)(price / sim::kPriceScale),
        (long long)(price % sim::kPriceScale),
        shares, (unsigned long long)token);
}
```

Why is this a problem?

The `fprintf` parses the format string, converts seven values (including two 64-bit divisions for the fixed-point price), and copies everything into the file buffer. This costs ∼300–400 ns per call. The exchange does not need the formatted string, and neither does the strategy—the only consumer is a human reading the log after the run.

What is the fix?

Defer the formatting. `Record` pushes a small struct of raw integers into a preallocated vector (∼5–10 ns), and a `Flush()` method does all the `fprintf` work at end-of-run:

```
// decision_log.h -- CHANGED: capture raw data, format later
struct Entry {
    sim::Timestamp now; sim::Price price; sim::Token token;
    uint32_t shares; char side; const char* action;
};

void Record(...) {
    Entry e{}; e.now = now; e.price = price; e.token = token;
    e.shares = shares; e.side = (char)side; e.action = action;
    entries_.push_back(e);  // ~5-10 ns, pre-reserved
}

void Flush() {  // called at end-of-run
    for (const Entry& e : entries_) {
        std::fprintf(file_, "%llu %s %c %lld.%04lld x %u ...\n",
            ...);  // all formatting happens here
    }
}
```

Why does this preserve write-ahead safety?

The design document says logging happens before sending so that if the process crashes mid-send, the log shows what was intended. The fix preserves this: the raw data is captured into the vector *before* `Send` is called. The original `fprintf` also wrote to an in-memory buffer (since `FILE*` is buffered), not directly to disk—so both versions have the data in memory before the send. Only the formatting is deferred; the data capture still precedes the send.

Net effect

Combined with the exception fix and a one-line `history_.reserve(256)` in `orders.h` (to eliminate vector reallocation spikes), `basic.scn` mean latency drops from 3,174 ns to 490 ns, and the median drops from 2,458 ns to 92 ns.

Here is the link to the doc I maintained to show my understanding of this assignment better: Google Doc link.